



Security Review For Meteora



Collaborative Audit Prepared For:
Lead Security Expert(s):

Meteora
bin2chen
carrotsmuggler
March 23 - April 27, 2026

Date Audited:

Introduction

DLMM is a decentralized liquidity management and swapping platform that offers users high flexibility in providing liquidity, open limit orders, and performing swaps. It utilizes a bin-based approach, where liquidity is distributed across different bins, each representing a price range. This allows for concentrated liquidity provisioning, thereby enhancing capital efficiency for liquidity providers.

Scope

Repository: [MetedoraAg/DLMM](#)

Audited Commit: [c83256f8cf22a7e32f7e7c61b052bdea7dd95090](#)

Final Commit: [559a27573ff6942e99cb358fbe21c4722dd39775](#)

Files:

- [libs/jupiter/Cargo.toml](#)
- [libs/jupiter/src/lib.rs](#)
- [libs/zap/Cargo.toml](#)
- [libs/zap/src/lib.rs](#)
- [programs/lb_clmm/src/constants.rs](#)
- [programs/lb_clmm/src/errors.rs](#)
- [programs/lb_clmm/src/events.rs](#)
- [programs/lb_clmm/src/fee_calculator.rs](#)
- [programs/lb_clmm/src/instructions/admin/close_bin_array.rs](#)
- [programs/lb_clmm/src/instructions/admin/close_claim_fee_operator_account.rs](#)
- [programs/lb_clmm/src/instructions/admin/close_operator_account.rs](#)
- [programs/lb_clmm/src/instructions/admin/create_operator_account.rs](#)
- [programs/lb_clmm/src/instructions/admin/mod.rs](#)
- [programs/lb_clmm/src/instructions/claim_fee.rs](#)
- [programs/lb_clmm/src/instructions/claim_reward.rs](#)
- [programs/lb_clmm/src/instructions/deposit/add_liquidity_by_strategy_one_side.rs](#)
- [programs/lb_clmm/src/instructions/deposit/add_liquidity_by_strategy.rs](#)
- [programs/lb_clmm/src/instructions/deposit/add_liquidity_by_weight_one_side.rs](#)
- [programs/lb_clmm/src/instructions/deposit/add_liquidity_by_weight.rs](#)

- `programs/lb_clmm/src/instructions/deposit/add_liquidity.rs`
- `programs/lb_clmm/src/instructions/deposit/add_liquidity_single_side_precise.rs`
- `programs/lb_clmm/src/instructions/deposit/mod.rs`
- `programs/lb_clmm/src/instructions/initialize_pool/initialize_customizable_permissionless_lb_pair.rs`
- `programs/lb_clmm/src/instructions/initialize_pool/initialize_permission_lb_pair.rs`
- `programs/lb_clmm/src/instructions/initialize_pool/initialize_permissionless_lb_pair.rs`
- `programs/lb_clmm/src/instructions/initialize_pool/mod.rs`
- `programs/lb_clmm/src/instructions/limit_order/cancel_limit_order.rs`
- `programs/lb_clmm/src/instructions/limit_order/close_limit_order_if_empty.rs`
- `programs/lb_clmm/src/instructions/limit_order/mod.rs`
- `programs/lb_clmm/src/instructions/limit_order/place_limit_order.rs`
- `programs/lb_clmm/src/instructions/mod.rs`
- `programs/lb_clmm/src/instructions/operator/close_preset_parameter2.rs`
- `programs/lb_clmm/src/instructions/operator/close_preset_parameter.rs`
- `programs/lb_clmm/src/instructions/operator/close_token_badge.rs`
- `programs/lb_clmm/src/instructions/operator/fund_reward.rs`
- `programs/lb_clmm/src/instructions/operator/initialize_preset_parameters.rs`
- `programs/lb_clmm/src/instructions/operator/initialize_reward.rs`
- `programs/lb_clmm/src/instructions/operator/initialize_token_badge.rs`
- `programs/lb_clmm/src/instructions/operator/mod.rs`
- `programs/lb_clmm/src/instructions/operator/set_activation_point.rs`
- `programs/lb_clmm/src/instructions/operator/set_pair_status.rs`
- `programs/lb_clmm/src/instructions/operator/set_pre_activation_duration.rs`
- `programs/lb_clmm/src/instructions/operator/set_pre_activation_swap_address.rs`
- `programs/lb_clmm/src/instructions/operator/update_fee_parameters.rs`
- `programs/lb_clmm/src/instructions/operator/update_reward_duration.rs`
- `programs/lb_clmm/src/instructions/operator/update_reward_funder.rs`
- `programs/lb_clmm/src/instructions/operator/withdraw_ineligible_reward.rs`
- `programs/lb_clmm/src/instructions/operator/withdraw_protocol_fee.rs`
- `programs/lb_clmm/src/instructions/operator/zap_protocol_fee/damm_v2_zap.rs`

- `programs/lb_clmm/src/instructions/operator/zap_protocol_fee/dlmm_zap.rs`
- `programs/lb_clmm/src/instructions/operator/zap_protocol_fee/jup_v6_zap.rs`
- `programs/lb_clmm/src/instructions/operator/zap_protocol_fee/mod.rs`
- `programs/lb_clmm/src/instructions/operator/zap_protocol_fee/process_zap_protocol_fee.rs`
- `programs/lb_clmm/src/instructions/permissionless_operations/go_to_a_bin.rs`
- `programs/lb_clmm/src/instructions/permissionless_operations/increase_oracle_length.rs`
- `programs/lb_clmm/src/instructions/permissionless_operations/initialize_bin_array_bitmap_extension.rs`
- `programs/lb_clmm/src/instructions/permissionless_operations/initialize_bin_array.rs`
- `programs/lb_clmm/src/instructions/permissionless_operations/mod.rs`
- `programs/lb_clmm/src/instructions/position_authorize.rs`
- `programs/lb_clmm/src/instructions/position/close_position.rs`
- `programs/lb_clmm/src/instructions/position/common.rs`
- `programs/lb_clmm/src/instructions/position/decrease_position_length.rs`
- `programs/lb_clmm/src/instructions/position/increase_position_length2.rs`
- `programs/lb_clmm/src/instructions/position/increase_position_length.rs`
- `programs/lb_clmm/src/instructions/position/initialize_position2.rs`
- `programs/lb_clmm/src/instructions/position/initialize_position_by_operator.rs`
- `programs/lb_clmm/src/instructions/position/initialize_position_pda.rs`
- `programs/lb_clmm/src/instructions/position/initialize_position.rs`
- `programs/lb_clmm/src/instructions/position/mod.rs`
- `programs/lb_clmm/src/instructions/position/set_permissionless_operation_bits.rs`
- `programs/lb_clmm/src/instructions/position/update_position_operator.rs`
- `programs/lb_clmm/src/instructions/rebalance/mod.rs`
- `programs/lb_clmm/src/instructions/rebalance/process_add_liquidity.rs`
- `programs/lb_clmm/src/instructions/rebalance/process_claim_fee_and_rewards.rs`
- `programs/lb_clmm/src/instructions/rebalance/process_remove_liquidity.rs`
- `programs/lb_clmm/src/instructions/rebalance/process_resize_position.rs`
- `programs/lb_clmm/src/instructions/rebalance/rebalance_params.rs`
- `programs/lb_clmm/src/instructions/rebalance/rebalance_wrapper.rs`

- programs/lb_clmm/src/instructions/set_pair_status_permissionless.rs
- programs/lb_clmm/src/instructions/swap.rs
- programs/lb_clmm/src/instructions/update_fees_and_rewards.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity2.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity_by_strategy2.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity_by_weight2.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity_single_side_precise2.rs
- programs/lb_clmm/src/instructions/v2/claim_fee2.rs
- programs/lb_clmm/src/instructions/v2/claim_reward2.rs
- programs/lb_clmm/src/instructions/v2/close_position2.rs
- programs/lb_clmm/src/instructions/v2/close_position_if_empty.rs
- programs/lb_clmm/src/instructions/v2/initialize_customizable_permissionless_lb_pair2.rs
- programs/lb_clmm/src/instructions/v2/initialize_permissionless_lb_pair2.rs
- programs/lb_clmm/src/instructions/v2/mod.rs
- programs/lb_clmm/src/instructions/v2/remove_liquidity2.rs
- programs/lb_clmm/src/instructions/v2/remove_liquidity_by_range2.rs
- programs/lb_clmm/src/instructions/v2/swap2.rs
- programs/lb_clmm/src/instructions/v2/update_fees_and_reward2.rs
- programs/lb_clmm/src/instructions/withdraw/mod.rs
- programs/lb_clmm/src/instructions/withdraw/remove_all_liquidity.rs
- programs/lb_clmm/src/instructions/withdraw/remove_liquidity_by_range.rs
- programs/lb_clmm/src/instructions/withdraw/remove_liquidity.rs
- programs/lb_clmm/src/lb_access_control.rs
- programs/lb_clmm/src/lib.rs
- programs/lb_clmm/src/math/bin_math.rs
- programs/lb_clmm/src/math/mod.rs
- programs/lb_clmm/src/math/price_math.rs
- programs/lb_clmm/src/math/safe_math.rs
- programs/lb_clmm/src/math/u128x128_math.rs
- programs/lb_clmm/src/math/u64x64_math.rs
- programs/lb_clmm/src/math/utils_math.rs

- `programs/lb_clmm/src/math/weight_to_amounts.rs`
- `programs/lb_clmm/src/pair_action_access/base.rs`
- `programs/lb_clmm/src/pair_action_access/customizable_permissionless_pair.rs`
- `programs/lb_clmm/src/pair_action_access/mod.rs`
- `programs/lb_clmm/src/pair_action_access/permissionless_pair.rs`
- `programs/lb_clmm/src/pair_action_access/permission_pair.rs`
- `programs/lb_clmm/src/state/bin_array_bitmap_extension.rs`
- `programs/lb_clmm/src/state/bin.rs`
- `programs/lb_clmm/src/state/claim_fee_operator.rs`
- `programs/lb_clmm/src/state/dynamic_position.rs`
- `programs/lb_clmm/src/state/lb_pair.rs`
- `programs/lb_clmm/src/state/limit_order.rs`
- `programs/lb_clmm/src/state/mod.rs`
- `programs/lb_clmm/src/state/operator.rs`
- `programs/lb_clmm/src/state/oracle.rs`
- `programs/lb_clmm/src/state/parameters.rs`
- `programs/lb_clmm/src/state/position.rs`
- `programs/lb_clmm/src/state/preset_parameters2.rs`
- `programs/lb_clmm/src/state/preset_parameters.rs`
- `programs/lb_clmm/src/state/token_badge.rs`
- `programs/lb_clmm/src/utils/alpha_vault.rs`
- `programs/lb_clmm/src/utils/create_pda_account.rs`
- `programs/lb_clmm/src/utils/mod.rs`
- `programs/lb_clmm/src/utils/remaining_accounts_util.rs`
- `programs/lb_clmm/src/utils/seeds.rs`
- `programs/lb_clmm/src/utils/token2022.rs`
- `programs/lb_clmm/src/utils/token.rs`
- `programs/lb_clmm/src/utils/transfer_memo.rs`

Final Commit Hash

559a27573ff6942e99cb358fbe21c4722dd39775

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	6	11

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: `go_to_a_bin()` lacks pool state checks and oracle updates [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/129>

Vulnerability Detail

`go_to_a_bin()` only checks whether liquidity is empty. The following validations and updates are missing:

1. Missing pool state validation:
 - Before liquidity is added, the presale price may be adjusted by anyone.
 - `active_id` can still be updated while the pool is disabled.
2. Missing oracle synchronization before the jump:
 - Time before the jump can be attributed to the new bin.
 - Oracle state can report the price as staying at a bin that was never reached through a swap.
3. Lack of access control
 - anyone can maliciously move `active_id` to trigger additional bin crossings and increase `variable_fee`

Impact

Unexpected active-bin changes before pool activation. The pool's price state can be updated without the accounting side effects normally associated with price movement. This can lead to stale oracle observations and incorrect `variable_fee`.

Recommendation

Restrict this instruction to the pool creator or an admin

Discussion

bin2chen66

codewithgun: Fixed Missing Pool State Checks in <https://github.com/MeteoraAg/DLMM/pull/513>. We acknowledge for No Oracle/Reference Update Before the Jump

bin2chen66

<https://github.com/MeteoraAg/DLMM/pull/513>

updated:

- CustomizablePermissionlessLbPair/PermissionLbPair: (enabled && activated) Or GoToABin_Operator
- PermissionlessLbPair:enabled Or GoToABin_Operator

Note: This has a dual effect: it can be used to move the `active_id` across empty bins to reduce bin crossings during swaps, but it can also be abused to increase bin crossings.

Issue M-2: Permissionless fee claims can cause users to be maliciously charged higher transfer fees [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/130>

Vulnerability Detail

Positions can opt into permissionless fee claiming through `CLAIM_FEE_MASK`. When this bit is enabled, any caller can trigger `claim_fee2`, while the claimed tokens are still sent to the position owner or configured fee owner. This is generally safe for ordinary tokens because the caller cannot redirect the funds. However, Token-2022 transfer-fee mints can charge a fee on every transfer, often with a per-transfer cap.

For example, assume a token supports `TransferFee`, with `maximum_fee = 5` and `fee_points = 1%`.

1. Claiming 1,000 tokens in a single transaction incurs a fee of 5 because of the cap.
2. Claiming 1,000 tokens across 10 transactions incurs a total fee of 10 ($1 * 10$).

Impact

An attacker can repeatedly trigger small claims or claim narrow bin ranges so that the receiver pays multiple transfer fees. With capped transfer fees, splitting one large transfer into multiple smaller transfers can result in a higher total fee than a single claim.

Recommendation

Implement a whitelist mechanism or when enabling `CLAIM_FEE_MASK`, include a warning about this risk in the UI or documentation.

Discussion

bin2chen66

codewithgun: Fixed in <https://github.com/MeteoraAg/DLMM/pull/512>

bin2chen66

Fixed in <https://github.com/MeteoraAg/DLMM/pull/512>

included a warning about this risk in documentation

Issue M-3: rebalance_liquidity() may skip the min_withdraw_amount check [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/131>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

`rebalance_liquidity()` nets removed liquidity, composition fees, and newly added liquidity before determining whether tokens should be transferred to or from the user.

The `min_withdraw_amount` slippage check is only executed in the branch where `removed_amount > added_amount`. If the rebalance reuses all removed tokens for the add side, or if removed and added amounts are equal, the branch is skipped and the user's minimum-withdrawal constraint is never enforced.

For example:

- A user executes `rebalance_liquidity()`.
 - The rebalance performs adds, removes, and fee accounting.
 - `min_withdraw_amount = 10`
 - `max_deposit_amount = 0`

Expected result: **the user should receive at least 10.**

However, in the final on-chain execution, `removed_amount == added_amount`, so this check is skipped.

```
fn process_rebalance_transfer<'a, 'c: 'info, 'info>(  
    ....  
    // Total withdrawn amount > amount user wish to deposit. The pool will refund  
    ↪ the remaining amount after take consideration into reusing withdrawn amount  
    ↪ for deposit.  
    if removed_amount > added_amount {  
        let withdraw_amount = removed_amount.safe_sub(added_amount)?;  
        let memo_transfer_ctx = MemoTransferContext {  
            memo_program,  
            memo: REMOVE_LIQUIDITY_AND_FEE_MEMO,  
        };  
        let excluded_fee_withdraw_amount =  
            calculate_transfer_fee_excluded_amount(token_mint,  
            ↪ withdraw_amount)?.amount;  
        require!(  
@>>>        excluded_fee_withdraw_amount >= min_withdraw_amount,
```

```
        LLError::ExceededAmountSlippageTolerance
    );
```

Impact

The `min_withdraw_amount` slippage check can be skipped.

Recommendation

Place it outside the `if` statement, for example:

```
fn process_rebalance_transfer<'a, 'c: 'info, 'info>(
    withdrawn_amount: u64,
    fee_amount: u64,
    added_amount: u64,
    accounts: RebalanceTransferAccounts<'a, 'info>,
    transfer_hook_accounts: Option<&'c [AccountInfo<'info>]>,
    min_withdraw_amount: u64,
    max_deposit_amount: u64,
) -> Result<()> {

    + let excluded_fee_withdraw_amount = 0
    + let included_fee_deposit_amount = 0
      if removed_amount > added_amount {
          ...
      } else if added_amount > removed_amount {
          ...
      }

    + require!(
    +     excluded_fee_withdraw_amount >= min_withdraw_amount,
    +     LLError::ExceededAmountSlippageTolerance
    + );
    + require!(
    +     included_fee_deposit_amount <= max_deposit_amount,
    +     LLError::ExceededAmountSlippageTolerance
    + );
```

Discussion

codewithgun

We acknowledge this. By design choice, `min_withdraw_amount` and `max_deposit_amount` are delta amount in rebalancing, not absolute amount

bin2chen66

hey @codewithgun

Thanks for your reply.

By “delta amount”, do you mean the minimum net amount the user expects to receive after rebalancing?

For example, if a user calls `rebalance_liquidity(min_withdraw_amount = 10)`, the expectation would be to **receive at least 10 tokens net** after the transaction is successfully executed..

However, if the final execution results in `removed_amount == added_amount`, the check is skipped, the transaction still succeeds, and the user **ends up receiving 0 tokens**.

This seems inconsistent with user expectations.

Would it be possible to enforce the check more strictly, even in this case?

codewithgun

hey @codewithgun

Thanks for your reply.

By “delta amount”, do you mean the minimum net amount the user expects to receive after rebalancing?

For example, if a user calls `rebalance_liquidity(min_withdraw_amount = 10)`, the expectation would be to **receive at least 10 tokens net**.

However, if the final execution results in `removed_amount == added_amount`, the check is skipped, the transaction still succeeds, and the user **ends up receiving 0 tokens**.

This seems inconsistent with user expectations.

Would it be possible to enforce the check more strictly, even in this case?

Ah sorry, what I'm trying to say is the `min_withdraw_amount`, and `max_deposit_amount` based on delta amount a.k.a `removed_amount > added_amount || added_amount > removed_amount` so the parameter is only effective when there's delta between add and remove.

Therefore, it's a design choice and we will acknowledge this.

Issue M-4: Exact-out swaps may underestimate excluded_fee_amount_in across segmented liquidity [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/132>

Vulnerability Detail

The exact-out path computes the required input amount once for the aggregate requested output. This is correct when the bin behaves as one continuous liquidity source at the same rounding boundary. However, the bin can fill output across multiple internal segments, such as market-maker liquidity and limit-order liquidity. Each segment can round input requirements independently.

For integer arithmetic, $\text{ceil}((a + b) / p)$ can be smaller than $\text{ceil}(a / p) + \text{ceil}(b / p)$. Therefore, the aggregate input estimate can be lower than the actual input required by the segmented execution path. The later exact-in simulation can fail to produce the requested output and revert.

Example:

- price = 0.4, MM has 1Y, LO has 1Y, and the user requests exact out 2Y.
- The code computes input in aggregate: $\text{excluded_fee_amount_in} = \text{ceil}(2 / 0.4) = 5X$.
- Actual segmented execution needs: $\text{ceil}(1 / 0.4) + \text{ceil}(1 / 0.4) = 3X + 3X = 6X$.

Therefore, input is **undercounted by 1X**. The later exact-in simulation cannot output 2Y, and the swap reverts.

Note: with only LO liquidity, a similar issue can still occur if both `processed_order_remaining_amount` and `open_order_amount` are required.

- DLMM/programs/lb_clmm/src/state/bin.rs

```
pub fn get_swap_result_exact_out(
...
    if included_fee_amount_out >= max_amount_out {
...
    } else {
        // included_fee_amount_out < max_amount_out, then we must fulfill exact
        // out in that round
@>> let excluded_fee_amount_in = Bin::get_amount_in(
            self.price,
            included_fee_amount_out,
            swap_for_y,
```

```

        Rounding::Up,
    )?;
    let included_fee_amount_in = if fee_mode.fees_on_input {
        let (included_fee_amount_in, _) =
            get_included_fee_amount(trading_fee_numerator,
                ↪ excluded_fee_amount_in)?;
        included_fee_amount_in
    } else {
        excluded_fee_amount_in
    };

```

Impact

Some exact-out swaps can unexpectedly revert even when the user's maximum input would have been sufficient for the segmented fill.

Recommendation

When using both MM and LO liquidity, calculate the required input for each segment separately and then combine the results.

Discussion

codewithgun

Fixed in <https://github.com/MeteoraAg/DLMM/pull/522>

bin2chen66

Fixed in <https://github.com/MeteoraAg/DLMM/pull/522>

Add a new method `get_excluded_fee_amount_in()` to calculate `excluded_fee_amount_in`, consuming in order: `mm` → `processed_order_remaining_amount` → `open_order_amount`.

Issue M-5: Same-bin swaps repeatedly decay volatility, enabling dynamic-fee bypass [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/142>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`last_update_timestamp` is only advanced when a swap crosses bins. Same-bin swaps leave it stale, so each subsequent swap re-enters the decay branch in `update_references`. A few dust-sized same-bin swaps zero the volatility accumulator and erase the dynamic fee on the next large trade.

Vulnerability Detail

In `swap2.rs`, the timestamp is only written on a bin cross:

```
if start_active_id != lb_pair.active_id {
    lb_pair.v_parameters.last_update_timestamp = current_timestamp;
}
```

The canonical update inside `update_references` is commented out:

```
// self.last_update_timestamp = current_timestamp;
```

`update_references` then runs:

```
let elapsed = current_timestamp.safe_sub(self.last_update_timestamp)?;
if elapsed >= static_params.get_filter_period() as i64 {
    self.index_reference = active_id;
    if elapsed < static_params.get_decay_period() as i64 {
        self.volatility_reference = self.volatility_accumulator
            .safe_mul(static_params.reduction_factor as u32)?
            .safe_div(BASIS_POINT_MAX as u32)?;
    } else {
        self.volatility_reference = 0;
    }
}
```

For a same-bin swap, `delta_id = 0`, so `update_volatility_accumulator` writes `volatility_accumulator = volatility_reference`. Each call multiplies the accumulator by `reduction_factor / BASIS_POINT_MAX` (5% with default params).

Normally many swaps in one block do not affect volatility because `last_update_timestamp` is bumped on the first swap and the rest see `elapsed < filter_period`. With the bin-cross gate, a same-bin swap leaves the timestamp stale and every follow-up swap re-applies the 5% decay.

Trace with `filter_period = 30s`, `reduction_factor = 500`, `last_update_timestamp = 1000`, accumulator `V`:

- `t=1031`, dust swap, no bin cross: `elapsed=31 > 30` -> accumulator = `0.05V`.
- `t=1031`, dust swap: timestamp still 1000, `elapsed=31` -> accumulator = `0.0025V`.
- A few more such swaps drive the accumulator to 0 within the same block.

The attacker then submits the real swap with zero variable fee.

Impact

The variable fee that protects LPs in volatile markets can be erased on demand with a handful of dust swaps. LPs lose the volatility premium on the subsequent large trade.

Recommendation

Update `last_update_timestamp` on every swap, not only on bin crosses.

Discussion

codewithgun

We acknowledge this because update `last_update_timestamp` on every swap will allow attacker to keep pool fees at max by swapping dust amount to keep `elapsed_seconds_since_last_update < filter_period`

Issue M-6: Limit-order fees mix across ask/bid generations on side flips [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/143>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`place_limit_order` zeroes `total_processing_order_amount` and bumps `order_age` when a bin's limit-order side flips, but does not reset the per-side fulfilled and fee accumulators. After an ask $\rightarrow$ bid $\rightarrow$ ask cycle, fills from the new ask epoch share the same `limit_order_fee_ask_side` and `fulfilled_order_amount_x` pools as the old ask epoch. Pro-rata distribution then redistributes fees between users from different epochs at different fee rates.

Vulnerability Detail

The flip branch in `bin.rs::place_limit_order`:

```
if user_ask_side != bin_ask_side && self.total_processing_order_amount > 0 {  
    self.total_processing_order_amount = 0;  
    self.order_age = self.order_age.safe_add(1)?;  
}
```

`fulfilled_order_amount_x/y` and `limit_order_fee_ask_side/bid_side` are not cleared. They are only zeroed in `apply_cancel_limit_order`'s "all users cancelled" cleanup.

`calculate_limit_order_fee` divides the merged ask-side accumulators pro-rata:

```
let (limit_order_fee, _) = calculate_shared_amounts(  
    self.limit_order_fee_ask_side,  
    fulfilled_amount,  
    self.fulfilled_order_amount_x,  
    Rounding::Down,  
)?;
```

If a user from an earlier ask epoch has not yet cancelled, fills from the new ask epoch are added on top. The pro-rata assumption "all `fulfilled_order_amount_x` contributors paid fees at the same rate" is violated across epochs.

Trace

A places 100 X. Swap fills A at fee rate 5%:

- fulfilled_x = 100, fee_ask = 5, order_age = 1.

B places bid (flip): total_processing = 0, order_age = 2. fulfilled_x and fee_ask untouched. Bid fills on Y side only.

C places ask (flip again): total_processing = 0, order_age bumped. fulfilled_x = 100, fee_ask = 5 still stale. Swap fills C's 100 X at higher dynamic-fee rate, fee 20:

- fulfilled_x = 200, fee_ask = 25.

Cancellations:

- A: fee = $25 * 100 / 200 = 12$ (expected 5).
- C: fee = $13 * 100 / 100 = 13$ (expected 20).

A skims 7 of C's fees. Bin-level conservation holds; per-user fairness does not.

Impact

Old-epoch ask/bid users who delay cancellation skim fees from new same-side epochs; new-epoch users receive less fee than they paid. Requires a full ask->bid->ask cycle on the same bin without an intermediate cancel by the straggler.

Recommendation

On a side flip, reset the departing side's accumulators, or block the flip while the departing side still has unclaimed fulfilled amount.

Discussion

codewithgun

We acknowledge this

Issue L-1: Permissioned pool initialization silently ignores `ix_data.protocol_share` [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/127>

Vulnerability Detail

The permissioned pool initialization instruction data includes a `protocol_share` field, but the handler does not use it when constructing `PoolPresetParameter`. Instead, it always writes the constant `PROTOCOL_SHARE`. This creates a mismatch between the instruction interface and the resulting pool configuration.

```
pub fn handle(
    ctx: Context<InitializePermissionLbPair>,
    ix_data: InitPermissionPairIx,
) -> Result<()> {
    ...

    let preset_parameter = PoolPresetParameter {
        bin_step: ix_data.bin_step,
        base_factor: ix_data.base_factor,
        min_bin_id,
        max_bin_id,
        base_fee_power_factor: ix_data.base_fee_power_factor,
@>> protocol_share: PROTOCOL_SHARE,
        function_type: function_type.into(),
        collect_fee_mode: ix_data.collect_fee_mode,
        // Default on purpose. This is to disable variable fee parameters
        ..Default::default()
    };
    preset_parameter.validate_initialize()?;
```

Impact

Pool configuration can differ from operator expectations, and off-chain integrations may assume the provided value was applied.

Recommendation

Remove `InitPermissionPairIx.protocol_share` or add a note stating that the field is ignored.

Discussion

bin2chen66

codewithgun: Fixed in <https://github.com/MeteoraAg/DLMM/pull/509>

bin2chen66

Fixed in <https://github.com/MeteoraAg/DLMM/pull/509>

Remove the `protocol_share` parameter (rename it to `padding0`).

Issue L-2: Fee update event omits base_fee_power_factor [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/128>

Vulnerability Detail

handle_update_base_fee updates three base-fee parameters: protocol_share, base_factor, and base_fee_power_factor. The emitted FeeParameterUpdate event includes only protocol_share and base_factor.

```
emit_cpi!(FeeParameterUpdate {  
    lb_pair: ctx.accounts.lb_pair.key(),  
    protocol_share: fee_parameter.protocol_share,  
    base_factor: fee_parameter.base_factor  
});
```

Impact

Off-chain indexers, monitoring systems, and analytics pipelines that rely on events will not observe changes to base_fee_power_factor.

Recommendation

Add base_fee_power_factor to the event struct and populate it when emitting the event.

Discussion

bin2chen66

codewithgun: Fixed in <https://github.com/MeteoraAg/DLMM/pull/508>

bin2chen66

Fixed in <https://github.com/MeteoraAg/DLMM/pull/508>

FeeParameterUpdate event now includes base_fee_power_factor

Issue L-3: Oracle expansion does not take effect immediately [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/133>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

Oracle expansion do not immediately use the newly added slots. The oracle keeps overwriting old initialized entries first.

Example:

- Before expansion: `length = 100, active_size = 100, idx = 50`.
- After expansion: `length = 200, active_size = 100, idx = 50`.
- The next sample goes to index 51, which is already initialized, so it overwrites old data.
- The new slots 100..199 are not used until the index eventually reaches them.

Impact

The expanded oracle capacity is delayed rather than effective immediately, so historical retention is temporarily smaller than users would reasonably expect after expansion.

Recommendation

Consider reordering the old data and setting `idx = active_size - 1`. However, this makes the program slightly more complex.

Discussion

bin2chen66

codewithgun: We acknowledge this

Issue L-4: Reinitializable PresetParameter2 can cause pools to use unexpected parameters [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/134>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

A `preset_parameter2` account can be closed and re-initialized at the same address with different values. `initialize_lb_pair2()` trusts the preset's current contents and does not require the user to confirm key fields such as `bin_step`.

Example:

- A user submits a transaction to create a pool with preset P, expecting `bin_step = 1`, and to add liquidity at `active_id = 10000` with range `[-100, +100]`, expecting price range A.
- The operator closes P.
- The operator re-initializes P with `bin_step = 20`.
- The operator's transaction executes first.
- The user's transaction executes afterward.
- The pool is created with `bin_step = 20`.
- The same liquidity parameters now map to price range B, not A.

```
#[access_control(is_valid_operator_role(&ctx.accounts.operator,  
  ↳ ctx.accounts.signer.key, OperatorPermission::ClosePresetParameter))]  
pub fn close_preset_parameter2(ctx: Context<ClosePresetParameter2>) ->  
  ↳ Result<()> {  
    ↳ Ok()  
  }  
  
pub struct InitializeLbPair2<'info> {  
  ...  
  pub preset_parameter: AccountLoader<'info, PresetParameter2>,  
  ...  
}
```

Impact

Pools can be initialized with parameters that differ from user expectations, causing liquidity to be added at an unexpected price range.

Recommendation

Operationally prevent administrators from recreating a preset at the same address with different parameters. Prefer enforcing this on-chain by making presets immutable, versioned, or validated against a user-provided expected parameter hash.

Discussion

bin2chen66

codewithgun: We acknowledge this. Currently, it's trusted operator, we will move it to multisig later

Issue L-5: `increase_position_length2()` applies an incorrect maximum-length check [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/135>

Vulnerability Detail

When `increase_position_length2()` is executed, it checks `length_to_add <= MAX_RESIZE_LENGTH_USIZE == 91`.

However, considering `DEFAULT_BIN_PER_POSITION_USIZE = 70`, the actual amount of account space that needs to be expanded can be smaller than `length_to_add`.

For example:

```
lower = 1
upper = 10
length_to_add = 100
```

The actual required expansion is only: $100 + 10 - 70 = 40$

```
pub fn handle(ctx: Context<IncreasePositionLength2>, minimum_upper_bin_id: i32) ->
    Result<> {
    ...
    require!(
        (length_to_add as usize) <= MAX_RESIZE_LENGTH_USIZE,
```

Note: `increase_position_length()` has a similar issue. In v1, if the requested length is too large, `PositionV2::new_space_after_add()` returns an error before the intended `LError::InvalidResizeLength` can be triggered.

Impact

The resize check is overly restrictive and can force users to execute the method multiple times.

Recommendation

Subtract the free capacity provided by the default bins before applying the maximum resize check.

Discussion

codewithgun

Fixed in <https://github.com/MeteoraAg/DLMM/pull/520> +
<https://github.com/MeteoraAg/DLMM/pull/521> (patch for bug found in
<https://github.com/MeteoraAg/DLMM/pull/520>)

bin2chen66

Fixed in <https://github.com/MeteoraAg/DLMM/pull/521>

Update the condition to: if `new_length > DEFAULT_BIN_PER_POSITION`, then `new_length - max(current_length, DEFAULT_BIN_PER_POSITION) <= MAX_RESIZE_LENGTH`

Note: In `increase_position_length()`, the `Anchor realloc = PositionV2::new_space_after_add(...)` is executed during the account validation phase.

As a result, oversized resize attempts are typically rejected by the Anchor/Solana `realloc` limits first, rather than reaching the custom `InvalidResizeLength` check.

Issue L-6: update_earning_per_token_stored_single_bin() lacks position-range validation [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/136>

Vulnerability Detail

update_earning_per_token_stored_single_bin() writes fee and reward accounting for a single bin. The comment states that no position-range check is needed because callers already use get_non_overlap_modify_bin_ids_bounded_by_position, but that is not true for every caller. this method is also called by add_liquidity2(), remove_liquidity2(), these external paths do not check whether bin_id is within the position range.

- DLMM/programs/lb_clmm/src/state/dynamic_position.rs

```
pub fn update_earning_per_token_stored_single_bin(
    &mut self,
    reward_flags: &[bool; NUM_REWARDS_USIZE],
    bin: &Bin,
    bin_id: i32,
) -> Result<()> {
    // No need to check whether bin_id is within position range or not because
    // ↳ get_non_overlap_modify_bin_ids_bounded_by_position function already checked
    // ↳ it
    let idx = get_idx_by_id(
        bin_id,
        self.global_data.lower_bin_id,
        self.outer_lower_bin_id,
    )?;

    let liquidity_share = self.get_liquidity_share(&idx);
```

Impact

It is still possible to maliciously specify a bin_id that falls outside the position range (upper_bin_id < ghostBinId < outer_lower_bin_id), resulting in dirty data being written to rewardPerTokenCompletes.

Recommendation

Add a position-range check in update_earning_per_token_stored_single_bin()

Discussion

bin2chen66

codewithgun: Fixed in <https://github.com/MeteoraAg/DLMM/pull/510>

bin2chen66

Fixed in <https://github.com/MeteoraAg/DLMM/pull/510>

All three call paths are now properly checked:

1. `rebalance_liquidity()`: via `get_non_overlap_modify_bin_ids_bounded_by_position()`
2. `add_liquidity2()`: via `position.id_within_position(bin_id)`
3. `remove_liquidity2()`: via `position.id_within_position(bin_id)`

Issue L-7: Composition fee round down maybe undercharge active-bin deposits [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/137>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

The composition fee is intended to charge users who deposit an imbalanced amount into the active bin, because such a deposit is economically similar to performing a swap against the active-bin inventory. The calculation multiplies by the fee rate and then scales down by `FEE_DENOMINATOR2` using floor division.

Normal swap fee paths commonly round fees up to avoid zero-fee small trades. By rounding the composition fee down, small or precisely chosen active-bin deposits can pay less than the equivalent swap path.

This creates a fee discrepancy between economically similar actions.

Example:

- Pool reserves: 1000 X and 1000 Y, price: 1
- Fee rate: 0.1%
- User deposits: 500 X

Then:

- the composition fee is rounded down to 0
- `add_liquidity + remove_liquidity` returns 300 X + 200 Y
- This is effectively equivalent to swapping 200 X for 200 Y

But with a normal swap 200 X -> Y:

- `fee = ceil(200 * 0.1%) = 1`
- `output = 199 Y`

So the comparison is:

- `add + remove`: 200 Y
- normal swap: 199 Y

```
/// Compute composition fee. Composition_fee = fee_amount * (1 + total fee rate)
pub fn compute_composition_fee(&self, delta_amount: u64) -> Result<u64> {
    let total_fee_rate = self.get_total_fee()?;
    // total_fee_rate 1e9 unit
    let fee_amount = u128::from(delta_amount).safe_mul(total_fee_rate)?;
```

```

        let composition_fee =
            fee_amount.safe_mul(u128::from(FEE_DENOMINATOR).safe_add(total_fee_rate
                ↪ )?)?;
        // 1e9 unit * 1e9 unit = 1e18, scale back
@>> let scaled_down_fee =
    ↪ composition_fee.safe_div(u128::from(FEE_DENOMINATOR).pow(2))?;

        Ok(scaled_down_fee
            .try_into()
            .map_err(|_| LSError::TypeCastFailed)?)
    }

```

Impact

The pool and protocol can lose small amounts of fee revenue. Active-bin deposit paths can be slightly cheaper than swap paths.

Recommendation

Round up composition fees

Discussion

bin2chen66

codewithgun: We acknowledge this

Issue L-8: Weight-based active-bin deposits can unexpectedly revert due to division by zero [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/138>

Vulnerability Detail

The weight-to-amount conversion handles distributions that include the active bin by computing X-side and Y-side weights, then deriving scaling factors k_x and k_y . If the active bin currently contains only token X, the active-bin Y weight w_{y0} becomes zero. If the requested distribution contains only the active bin or no bins below the active bin, $total_weight_y$ remains zero. The function then divides by $total_weight_y$.

Example:

Current state:

- `active_id = 10`
- `amount_x = 100`
- `amount_y = 0`, so only **X** exists in the active bin.

Call:

```
add_liquidity_by_weight(  
    amount_x = 300,  
    weights = [(10, 5), (11, 5), (12, 5)]  
)
```

Flow:

- `liquidity_parameter.validate()` → passes
- Call:

```
to_amounts_into_bin(  
    active_id = 10,  
    amount_x = 100,  
    amount_y = 0,  
    total_amount_x = 300,  
    total_amount_y = 0,  
    weights = [(10,5), (11,5), (12,5)]  
)
```

Inside calculation:

- $w_y = 0$
- $total_weight_y = 0$

- $ky = \text{total_amount_y} / \text{total_weight_y}$, which causes a **division by zero**.

```
pub fn to_amount_both_side(
    ...
    // find k
    let ky = U256::from(total_amount_y)
        .safe_shl((SCALE_OFFSET * 2).into())?
    @>> .safe_div(total_weight_y)?;
```

Impact

Valid liquidity additions can unexpectedly revert, causing a denial of service for affected weight-based deposit strategies.

Recommendation

If total_weight_x or total_weight_y is zero, set the corresponding kx or ky to MAX when the corresponding input amount is also zero. Otherwise, return a clear invalid-input error.

Discussion

bin2chen66

codewithgun: Fixed in <https://github.com/MeteoraAg/DLMM/pull/507>

bin2chen66

Fixed in <https://github.com/MeteoraAg/DLMM/pull/507>

In the `Some(index) => branch`, if $\text{total_weight_x/y} == 0$, kx/ky is set to MAX.

Issue L-9: zap_protocol_fee() lacks validation for platform_fee_bps and the route plan [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/139>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

The `zap_protocol_fee()` checks the Zap program id, the Zap discriminator, the source account, the final destination account, the pre-balance, and the maximum swap amount. However, the Jupiter route payload validation only checks that every swap leg is allowlisted. It does not reject `platform_fee_bps` fields

As a result, an operator can steal funds by providing their account as `platform_fee_account`

In addition, the validation does not ensure that the route plan fully converges.

```
impl ZapInfoProcessor for ZapJupV6RouteInfoProcessor {
    fn validate_payload(&self, payload: &[u8]) -> Result<()> {
        let route_params = jupiter::client::args::Route::try_from_slice(payload)?;
        ensure_whitelisted_swap_leg(&route_params.route_plan)?;

        Ok(())
    }
}
```

Impact

operator steal funds

Recommendation

1. Reject routes where `platform_fee_bps != 0`.
2. Validate that the route plan fully converges.

Discussion

bin2chen66

codewithgun: We acknowledged it because it's a permissioned endpoint, and it will be deprecated in the next release

Issue L-10: close_bin_array is not callable [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/140>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The function `close_bin_array` requires every bin in the array to equal `Bin::default()` (all fields zero, including price). But `initialize_bin_array` calls `BinArray::fill_prices`, which writes a non-zero price into every bin via `get_price_from_id`. After initialization no bin ever satisfies `bin.eq(&Bin::default())`, so the close path always reverts.

Vulnerability Detail

`fill_prices` populates every bin's price at init time:

```
pub fn fill_prices(&mut self, bin_step: u16) -> Result<()> {
    let lower_bin_id = self.get_lower_bin_id()?;
    for i in 0..MAX_BIN_PER_ARRAY_USIZE {
        let id = lower_bin_id.safe_add(i as i32)?;
        self.bins[i].price = get_price_from_id(id, bin_step)?;
    }
    Ok(())
}
```

`close_bin_array` then checks each bin against the all-zero default:

```
// Extra safety check
for bin in bin_array.bins.iter() {
    require!(bin.eq(&Bin::default()), LError::UndeterminedError);
}
```

Since `Bin::default().price == 0` and post-init `bin.price != 0`, the `require` always fails. The intent of the check is to confirm the bin array carries no liquidity, but it inadvertently keys on a field that is set unconditionally during initialization.

Impact

`close_bin_array` is dead code: out-of-range bin arrays cannot be closed and their rent cannot be reclaimed.

Recommendation

Check liquidity-bearing fields instead of the full `Bin::default()` equality.

Discussion

codewithgun

We will acknowledge this. It's for us to close invalid bin array created in the past, will be removed on next release.

Issue L-11: LP mining rewards capped at first 15 bins per swap [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-03-meteora-dlmm-update-mar-23rd/issues/141>

Summary

`process_swap` only records the first `MAX_REWARD_BIN_SPLIT_USIZE = 15` bins crossed in a swap into `bins_with_liquidity_swapped`. Any additional bins consumed past the 15th contribute their liquidity to the swap but receive no farm rewards.

Vulnerability Detail

In `swap2.rs`:

```
if bins_with_liquidity_swapped.len() < MAX_REWARD_BIN_SPLIT_USIZE {  
    bins_with_liquidity_swapped.push(lb_pair.active_id);  
    bin_array_index_with_liquidity_swapped  
        .insert(bin_array_index_in_remaining_accounts, true);  
}
```

`distribute_rewards_to_swapped_bin` later splits accumulated rewards equally across this list. Bins 16+ are never inserted, so they are skipped entirely during distribution even though their liquidity was used to fill the swap.

The cap exists to bound CU usage, but on large swaps that traverse many bins this causes silent under-rewarding of active LPs in the tail bins.

Impact

LPs in bins past the 15th touched bin in a single swap miss out on that swap's reward share. This behaviour should be documented for users and LPs.

Recommendation

Document the 15-bin reward cap in user-facing docs so LPs understand that very wide swaps distribute rewards only to the first 15 bins crossed.

Discussion

codewithgun

Fixed in <https://github.com/MeteoraAg/DLMM/pull/518>

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.